Faculty of Engineering and Technology
Electrical and Computer Engineering Department

Computer Architecture
ENCS4370

Project No.2
Pipelined RISC Processor Design

Prepared by:

Labiba Sharia	**1220228**
Joud Thaher	**1221381**
Amir Al-Rashayda	**1222596**

Instructor:
Dr. Ayman Hroub

Section: 1
Date: 10/1/2025

Abstract

This report details the design, implementation, and verification of a 16-bit pipelined processor based on a custom Instruction Set Architecture (ISA). The ISA includes R-type, I-type, and J-type instructions. It supports logical (AND), arithmetic (ADD, SUB), memory access (LW, SW), and control flow instructions (JMP, CALL, RET, BEQ, BNE, FOR). The processor features eight 16-bit general-purpose registers, a 16-bit Program Counter (PC), and a 16-bit Return Register (RR) for storing return addresses during function calls. Two separate physical memories—one for instructions and one for data—are implemented with word-addressable organization.

A five-stage pipelined architecture, including fetch, decode, execute, memory access, and write-back stages, is employed to enhance performance by overlapping instruction execution. Key enhancements include performance monitoring registers to track execution metrics, such as the total number of executed instructions, load/store operations, ALU instructions, control instructions, clock cycles, and pipeline stall cycles. The ALU includes additional functionality to generate signals for branch outcomes, such as zero flag.

The design process involved creating a detailed Datapath and control path, with block diagrams and truth tables for control signal determination. RTL code was written for each instruction, and control signals were defined using Boolean equations. Comprehensive verification was conducted using testbenches and simulation tools to test a variety of instruction sequences. The simulation results confirmed the correctness and efficiency of the processor, validating its ability to execute all supported instruction types reliably and effectively within the pipelined architecture.

Table of contents

Abstract 2

Introduction 6

- **Processer Architecture** 6
- **Instruction Format** 6
- **Instruction Set** 7
- **Pipelined Processer stages** 8
- **Instructions stages** 8

Design and Implementation 9

- **RTL Design** 9
- **Single Cycle and Pipelined Datapaths** 14
- **Detailed Datapath** 15
- **Pipelined Control Units** 21
- **Test Case and Simulation** 25
- **Teamwork** 29
- **Conclusion** 30

Table of Figures

Figure1 : Single Cycle Processer Datapath..... 14
Figure2 : Pipelined Processer Datapath 15
Figure3 :test case..... 25
Figure4 :values in data memory..... 25
Figure5 simulation snapshots: 26
Figure6 :..... 28
Figure7 :..... 28

List of Tables

Table 1: Instruction set 7
Table2 : Main control unit control signals 21
Table3 : Main control unit Boolean expression 21
Table4 :ALU control unit Signals 22
Table5 :ALU control unit expressions 22
Table6 :PC control unit signals 23
Table7 : PC control unit Boolean expressions 23
Table8 : Hazard unit Boolean expressions 24

Introduction

Processer Architecture

The processor is a 16-bit architecture designed for efficient performance and simplicity. It features an instruction and word size of 16 bits, supported by eight 16-bit general-purpose integer registers for data handling. A 16-bit Program Counter (PC) manages the flow of instructions, while a 16-bit Return Register (RR) stores return addresses for function calls. The processor employs a Harvard architecture with two separate physical memories: one for instructions and the other for data, ensuring optimized access and processing. It supports three instruction types: R-type (Register Type) for register operations, I-type (Immediate Type) for operations involving immediate values, and J-type (Jump Type) for control flow changes. The memory is word-addressable, allowing direct access to 16-bit words. Additionally, the Arithmetic Logic Unit (ALU) includes a zero-signal flag to indicate whether the result of the last operation is zero, assisting in conditional processing and decision-making.

Instruction Format

The processor supports three types of instructions: R-type for register-based operations, I-type for operations involving immediate values, and J-type for jump or control flow instructions.

R-type

Opcode (15-12)	R_d (11-9)	R_s (8-6)	R_t (5-3)	Function (2-0)

I-type

Opcode (15-12)	R_s (11-9)	R_t (8-6)	Immediate (5-0)

J-type

Opcode (15-12)	9-bit offset (11-3)	Function (2-0)

Instruction Set

Table 1: Instruction set

Inst type	Inst	Meaning	Opcode	func
R	AND	Computes the bitwise AND of two registers	0000	000
	ADD	Computes the sum of two registers	0000	001
	SUB	Computes the difference between two registers	0000	010
	SLL	Shifts the bits in the first source Register left by the count in the second source register	0000	011
	SRL	Shifts the bits in the first source Register right by the count in the second source register	0000	100
I	ANDI	Computes the bitwise AND of a register and an immediate value	0010	X
	ADDI	Computes the sum of a register and an immediate value	0011	X
	LW	Loads a word from memory into a register	0100	X
	SW	Stores a word from a register into memory	0101	X
	BEQ	Branches if two registers are equal	0110	X
	BNE	Branches if two registers are not equal	0111	X
	FOR	Eexecutes a loop at the address in Reg (Rs) for the count in Reg (Rt), decrementing Reg (Rt) each iteration until zero	1000	X
J	JMP	Unconditionally jumps to a specified address	0001	000
	CALL	Calls a subroutine and saves the return address	0001	001
	RET	Returns from a subroutine using the saved return address	0001	010

Pipelined Processer stages

To enhance performance, the processor implements a 5-stage pipeline: fetch, decode, execute, memory access, and write-back. Each stage performs a specific function in processing instructions:

1. Fetch (IF): Retrieves the next instruction from instruction memory using the Program Counter (PC).
2. Decode (ID): Interprets the fetched instruction, identifying the opcode, source and destination registers, and any immediate values.
3. Execute (EX): Performs the operation specified by the instruction using the Arithmetic Logic Unit (ALU), which also generates necessary signals like zero, carry, and overflow.
4. Memory Access (MEM): Handles load and store operations, reading from or writing to data memory.
5. Write-Back (WB): Updates the destination register with the result of the operation performed in the execute stage.

This pipeline structure allows instructions to be processed concurrently, improving performance by enabling multiple instructions to be in different stages at the same time.

Instructions stages

Not all instructions require all five stages of the pipeline: some use fewer stages:

- Load Instructions: Require all five stages: IF (Instruction Fetch), ID (Instruction Decode), EX (Execution), MEM (Memory Access), and WB (Write Back).
- Store Instructions: Use four stages: IF, ID, EX, and MEM.
- ALU Instructions: Require four stages: IF, ID, EX, and WB.
- Branch Instructions: Typically use three stages: IF, ID, and EX. However, this can be reduced to two stages if the branch target is calculated during the ID stage.
- Jump Instructions: Need only two stages: IF and ID.

Design and Implementation

RTL Design

Table 2: RTL set

1. **R-type**		
AND & ADD & SUB & SLL & SRL	**Instruction Fetch**	Instruction ← MEM[PC] PC ← PC + 1
	Instruction Decode	Opcode ← inst[15:12] Rd← inst[11:9] Rs ← inst[8:6] Rt← inst[5:3] function ← inst[2:0]
	Execution	ALU_Result ← ALU_Operation(Rs, Rt, Function)
	Memory Access	Not required
	Write Back	RegFile(Rd) ← ALU_Result
2. **I-type**		
ANDI	**Instruction Fetch**	Instruction ← MEM[PC] PC ← PC + 1
	Instruction Decode	Opcode ← inst[15:12] Rs ← inst[11:9] Rt ← inst[8:6] Imm ← zero_extend(inst[5:0])
	Execution	ALU_Result ← Rs & Imm
	Memory Access	Not required
	Write Back	RegFile[Rt] ← ALU_Result
ADDI	**Instruction Fetch**	Instruction ← MEM[PC] PC ← PC + 1

	Instruction Decode	Opcode ← inst[15:12] Rs ← inst[11:9] Rt ← inst[8:6] Imm ← sign_extend(inst[5:0])
	Execution	ALU_Result ← Rs + Immediate
	Memory Access	Not required
	Write Back	RegFile[Rt] ← ALU_Result
LW	**Instruction Fetch**	Instruction ← MEM[PC]PC ← PC + 1
	Instruction Decode	Opcode ← inst[15:12] Rs ← inst[11:9] Rt ← inst[8:6] Imm ← sign_extend(inst[5:0])
	Execution	ALU_Result ← Rs + Immediate
	Memory Access	MemData ← MEM[ALU_Result]
	Write Back	RegFile[Rt] ← MemData
SW	**Instruction Fetch**	Instruction ← MEM[PC] PC ← PC + 1
	Instruction Decode	Opcode ← inst[15:12] Rs ← inst[11:9] Rt ← inst[8:6] Imm ← sign_extend(inst[5:0])

	Execution	ALU_Result ← Rs + Imm
	Memory Access	MEM[ALU_Result] ← RegFile[Rt]
	Write Back	Not required
BEQ	**Instruction Fetch**	Instruction ← MEM[PC]PC ← PC + 1
	Instruction Decode	Opcode ← inst[15:12] Rs ← inst[11:9] Rt ← inst[8:6] Imm ← sign_extend(inst[5:0])
	Execution	ALU_Result ← Rs – Rt // zero flag updated
	Memory Access	Not required
	Write Back	Not required
	***Branch Decision**	If (Rs == Rt) **then** PC ← PC + sign_extend(Imm) // Branch taken **Else** PC ← PC + 1 // Branch not taken
BNE	**Instruction Fetch**	Instruction ← MEM[PC] PC ← PC + 1
	Instruction Decode	Opcode ← inst[15:12] Rs ← inst[11:9] Rt ← inst[8:6] Imm← sign_extend(inst[5:0])
	Execution	ALU_Result ← Rs - Rt // zero flag updated

	Memory Access	Not required
	Write Back	Not required
	***Branch Decision**	If (Rs != Rt) **then** PC ← PC + sign_extend(Imm) // Branch taken **Else** PC ← PC + 1 // Branch not taken
FOR	**Instruction Fetch**	Instruction ← MEM[PC]PC ← PC + 1
	Instruction Decode	Opcode ← inst[15:12] Rs← inst[11:9] Rt ← inst[8:6]
	Execution	ALU_Result ← Rt - 1 // Decrement the loop counter
	Memory Access	Not required
	Write Back	RegFile[Rt] ← ALU_Result // Update loop counter
	***Calculation Decision**	If (Rt == 0) **then** PC ← PC + 1 // Exit loop **Else** PC ← Rs // Loop continues, jump to Rs
3. J-type		
JMP	**Instruction Fetch**	Instruction ← MEM[PC] PC ← PC + 1
	Instruction Decode	Opcode ← inst[15:12] Offset ← inst[11:3] Function ← inst[2:0]
	Execution	Target ← {PC [15:9], Offset} // Concatenate PC

		[15:9] and offset Jump PC ← target
	Memory Access	Not required
	Write Back	Not required
	***Jump**	PC ← target
CALL	**Instruction Fetch**	Instruction ← MEM[PC] PC ← PC + 1
	Instruction Decode	Opcode ← inst[15:12] Offset ← inst[11:3] Function ← inst[2:0]
	Execution	Target ← {PC [15:9], Offset} // Concatenate PC [15:9] and offset RR ← PC + 1 // Save return address
	Memory Access	Not required
	Write Back	Not required
	***Jump**	PC ← target
RET	**Instruction Fetch**	Instruction ← MEM[PC] PC ← PC + 1
	Instruction Decode	Opcode ← inst[15:12] Function ← inst[2:0]
	Execution	PC ← RegFile[RR] // Load return address from RR
	Memory Access	Not required
	Write Back	Not required

Single Cycle and Pipelined Datapaths

The single cycle Datapath was designed to execute each instruction in a single clock cycle. This design integrates all the necessary components, such as the instruction memory, data memory, registers, ALU, and control units, into a straightforward architecture. The instruction memory fetches the instruction, which is decoded by the control unit to generate signals guiding the operation. The registers store intermediate data, while the ALU performs arithmetic or logical operations. Data memory is used for load and store instructions, and the Program Counter (PC) ensures sequential execution or branching based on control flow instructions. Although simple and effective for small instruction sets, the single-cycle approach requires the clock cycle to be long enough to accommodate the slowest instruction, limiting overall efficiency.

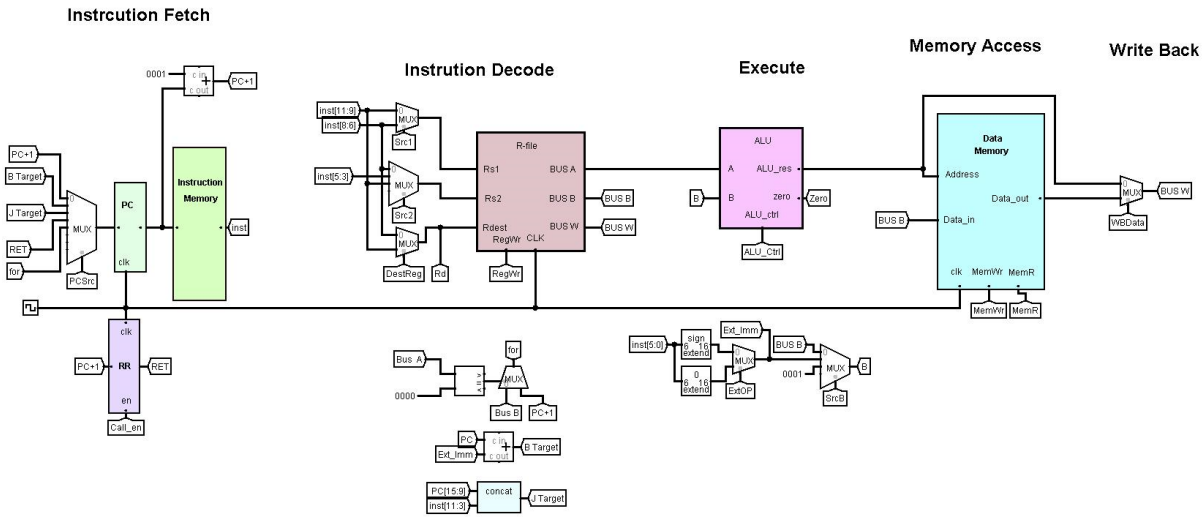

Figure1 : Single Cycle Processer Datapath

To improve performance, the single cycle Datapath was modified to incorporate pipelining, enabling overlapping execution of multiple instructions. The pipeline was divided into stages, such as instruction fetch, decode, execute, memory access, and write-back, pipeline registers (buffers) were added between each stage to store propagate results and control signals. This segmentation allows instructions to progress through different stages simultaneously, increasing instruction throughput. Additional control mechanisms, such as hazard detection units and forwarding paths, were implemented to address data and control hazards, ensuring correct operation. These modifications transform the Datapath into a highly efficient architecture capable of handling more complex workloads while maintaining high performance.

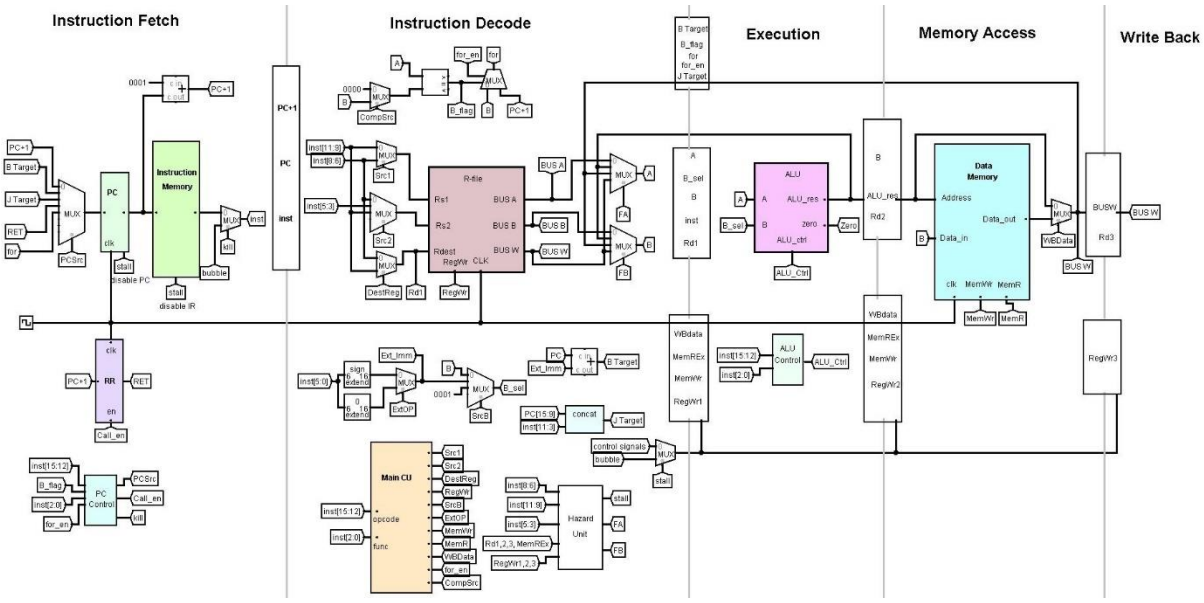

Figure2 : Pipelined Processer Datapath

Detailed Datapath

I. Instruction Fetch (IF)

1. **PC mux**
   In our data path, we use an 8-to-1 multiplexer (mux) to select the appropriate PC address based on the instruction. The mux is controlled by three selection lines: PCSrc0, PCSrc1, and PCSrc2.

   a. When the selection value is 000, the mux output is PC + 1, representing the next PC address.

   b. When the selection value is 001, the output is the branch target address, which is the current PC plus the sign-extended immediate value.

   c. When the selection value is 010, the output is the jump target address, formed by concatenating bits [15:9] of the current PC with a 9-bit offset.

   d. When the selection value is 011, the output is the value stored in the return register.

   e. When the selection value is 100, the output corresponds to the instruction target, which could be either PC + 1 or the value of the Rt register.

2. **Program Counter (PC)**
   The PC register stores the address of the next instruction to be retrieved from the instruction memory. It is updated at the start of each instruction fetch cycle to indicate the address of the following instruction.

3. **Return Register (RR)**
   The RR register temporarily stores the return address during function calls.
4. **Instruction Memory**
   The instruction memory stores the instructions to be executed by the processor. The instruction at the address specified by the PC is fetched during this stage.
5. **Adder**
   An adder is required: to calculate the next PC value (PC + 1), as each instruction is 16 bits and word-addressable (each word is 16 bits, equivalent to 1 byte).
6. **Stage Buffers**
   The stage buffers store the fetched instruction, the incremented Program Counter (PC) value for the next stage, and the original PC. These buffers facilitate smooth data transfer and synchronization between pipeline stages.
7. **Stall mux**
   The stall multiplexer (MUX) plays a critical role in maintaining the smooth operation of the pipeline by managing control hazards and data hazards. It determines whether the fetched instruction should proceed to the next pipeline stage or be replaced with a bubble (a no-operation instruction). The control signal Stall, generated by the hazard detection unit, governs the behaviour of this MUX:

   - When Stall = 0: The fetched instruction is forwarded to the next stage, allowing the pipeline to continue normal operation.
   - When Stall = 1: The MUX inserts a bubble into the pipeline, preventing the fetched instruction from advancing. This effectively freezes the pipeline, allowing earlier stages to resolve dependencies before new instructions are introduced.

   The stall MUX also interacts with the Program Counter (PC). When a stall is triggered, the PC is disabled (held constant) to ensure the same instruction is fetched again in subsequent cycles. This mechanism prevents incorrect instructions from entering the pipeline and ensures data consistency.

II. Instruction Decode (ID)

1. **Register File (R-file)**
   The Register File consists of eight 16-bit general-purpose registers. It supports two read buses (Bus A and Bus B) and one write bus (Bus W), enabling simultaneous reads from two registers and a write to one register. A write enable signal (RegWr), generated by the main control unit, determines whether data is written to the register file during the Write Back stage. The file includes three inputs to select the target registers: Rs1 selects the register to be read on Bus A, Rs2 selects the register to be read on Bus B, and Rdest selects the register to be written to.
2. **Rs1mux**
   This multiplexer determines the address of the first source register to be read on Bus A of the register file. It selects between inst [11:9], used as Rs in I-type

instructions, and inst[8:6], used as Rs in R-type instructions and Rt in (for) instruction. As a result, a 1-bit selection line (Src1) is needed, which is generated by the control unit based on the opcode and function.

3. **Rs2 mux**
   Likewise, the second mux determines the address of the second source register to be read on Bus B of the register file. It selects between inst [8:6] used as Rt in branch instructions (BEQ, BNE), inst[5:3] used as Rt in R-type instructions and inst[11:9] used as Rs in (for) instruction,. As a result, a 2-bit selection line (Src2) is needed, which is generated by the control unit based on the opcode.

4. **Rdest mux**
   The third multiplexer is responsible for selecting the address of the register to be written. It selects between inst[8:6], used as Rt in I-type instructions, and inst[11:9], used as Rd in R-type instructions. Consequently, a 1-bit selection line (DestReg) is required, which is generated by the control unit based on the opcode and function.

5. **For instruction mux and comparator**
   A comparator is required to compare the value of Rt with a constant zero to decide whether to exit the loop or continue. Additionally, a multiplexer is used to determine the next PC value for this instruction. If Rt = 0, the loop must exit, and the next PC will be PC + 1, selecting input 1 of the mux. Otherwise, the loop continues, and the next PC will be the loop target address (Rs), selecting input 0. The output of this mux serves as the input to the PC multiplexer.

   Note: If the instruction continues the loop, the EX and WB stages are required. However, if the loop exits, the EX, MEM, and WB stages are not needed**.**

6. **Adder**
   This adder computes the branch target address by adding the current PC to the sign-extended immediate value.

7. **Concatenation module**
   This module is used to compute the jump target address by concatenating bits [15:9] from PC with the 9-bit offset.

8. **Data Forwarding logic**
   It is an essential technique to resolve data hazards by enabling the direct use of results from previous instructions without waiting for them to be written back to the register file. Using specialized forwarding hardware, the result of an instruction is directly forwarded from the stage where it is produced to the stage where it is needed, such as the ALU input register or memory write port. To implement this, two multiplexers are added at this stage. These multiplexers receive data from the current stage (ID), EX stage, MEM stage,

and WB stage, enabling feedback through two control signals: FA and FB. The hardware handles common data dependencies hazard scenarios, ensuring smooth and efficient data flow across pipeline stages.

9. **Stage Buffers**
   The Stage Buffers temporarily store and transfer essential data and control signals between stages. The buffers used here are categorized into three types:

   1. Data Buffers: These hold critical information such as the current instruction, the first ALU input (A), the second ALU input (B_sel), the data to be written to memory (B), and the destination register (rd1). This data is passed from the Instruction Decode (ID) stage to the Execute (EX) stage.

   2. Forward Propagation Buffers: These buffers propagate the necessary control signals required for execution in subsequent stages which are which are WBdata, MemREx, MemWr, RegWr1.

   3. Backward Propagation Buffers: These handle the transmission of control signals and values needed for previous stages, such as the branch target (B target) and forwarding enable signal (for_en).

10. **Stall control Signals mux**
    This multiplexer determines whether the propagated control signals need to be stalled and replaced with a bubble or not.

11. **Hazard Handling**

    Pipeline hazards are managed efficiently to ensure smooth and correct execution of instructions. Two main types of hazards are addressed:

    1. Data Hazards: Data hazards occur when an instruction depends on the result of a previous instruction still in progress. These are handled by:

       - Stalling: If data dependencies cannot be resolved via forwarding, the hazard detection unit triggers a Stall signal. This signal freezes the PC and inserts a bubble (no-op instruction) into the pipeline, ensuring data consistency.

       - Forwarding: Specialized forwarding logic allows the direct use of data produced in previous stages (ID, EX, MEM, or WB) without waiting for it to be written back to the register file.

    2. Control Hazards: Control hazards arise from branch or jump instructions that alter the PC. These are managed by:

       - Bubble Insertion: Incorrectly fetched instructions following a branch or jump are replaced with bubbles to prevent their execution.

- PC Update: The PC is updated with the correct branch or jump target, ensuring the pipeline resumes at the proper instruction.

III. .Execute (EX)

1. **ALU**
   The ALU component has two inputs (A and B). It has a 4-bit selection line, which determines the operation to perform based on the opcode and function. It supports five operations: AND, ADD, SUB, SLL, and SRL. Additionally, the ALU generates a zero-flag used for branch instructions.

2. **B mux**
   The B multiplexer determines the second operand for the ALU based on the instruction type. When the input is 00, the instruction is R-type, and the second operand is Rt. For input 01, it handles I-type instructions (excluding for instructions) and selects the extended immediate as the second operand. Lastly, when the input is 10, it corresponds to the for instruction and selects a constant value of 1 as the second operand.

3. **Extender mux**
   The extender multiplexer decides the extension type for I-type instructions. If its input is 1, the instruction is logical and requires zero extension; otherwise, sign extension is applied for the rest of the I-type instructions. The output of the extender mux serves as the input to the second input of the B multiplexer.

4. **Stage Buffers**
   The buffers between the EX and MEM stages are:

   i. Data Buffers: the buffers buffers responsible for transmitting the values of Rd2, the ALU result (used as the address for the data memory or input for the WB data multiplexer), and B (which serves as the data input for the data memory).

   ii. Forward Propagation Buffers: These buffers propagate the necessary control signals required for execution in subsequent stages which are WBdata, MemREx, MemWr, RegWr2.

IV. Memory Access (MEM)

1. **Data Memory**
   The data memory module is a vital component of the processor's architecture, responsible for storing and retrieving data during program execution. It operates through various input and output signals to ensure efficient functionality. The

module receives an address input (Address) that specifies the memory location for reading or writing data, while the data input line (Data_in) provides the data to be stored at the specified address. Control signals dictate the memory operations: the Memory Write (MemWr) signal enables data from Data_in to be written to the designated address when active (for store instruction), and the Memory Read (MemR) signal triggers the reading of data from the specified address (for load instruction). The output of this module is delivered via the memory output line (Data_out), which provides the read data for use by other processor components.

2. **Write Back mux**
   The Write Back multiplexer selects the appropriate data to be written to the register file by choosing between the ALU result and the data read from the data memory (data_out). The output of this 2-to-1 multiplexer serves as the input to the write bus (Bus W) in the register file, ensuring the correct data is written based on the current operation. A 1-bit selection line (WBData), generated by the control unit, controls the multiplexer. When WBData is 0, the ALU result is written to the selected register; when WBData is 1, the data read from the data memory (data_out) is written instead.

3. **Stage Buffers**
   The buffers between the EX and MEM are:

   a. Data Buffers: buffers responsible for transmitting the values of Rd3, and Bus W which is the data to be written to the register file.

   b. Control Signal Buffers: These propagate necessary control signals from the MEM stage to the WB stage which is the RegWr3 signal.

V. Write Back (WB)

Bus W

The value propagated from MEM stage on Bus W is written to the register file to the destination register specified by the instruction.

Pipelined Control Units

Main control Unit Signals

The main control unit is responsible for generating control signals to coordinate the processor's operations. It takes a 4-bit opcode and a 3-bit function as inputs and generates various output signals. The control signals generated by this unit are summarized in the following table:

Table2 : Main control unit control signals

type	INST	Src1	Src2	DestReg	ExtOP	SrcB	RegWr	MemWr	MemR	WBData	for_en	CompSrc
R	AND	1	01	1	X	00	1	0	0	0	0	X
	ADD	1	01	1	X	00	1	0	0	0	0	X
	SUB	1	01	1	X	00	1	0	0	0	0	X
	SLL	1	01	1	X	00	1	0	0	0	0	X
	SRL	1	01	1	X	00	1	0	0	0	0	X
I	ANDI	0	X	0	1	01	1	0	0	0	0	X
	ADDI	0	X	0	0	01	1	0	0	0	0	X
	LW	0	X	0	0	01	1	0	1	1	0	X
	SW	0	X	X	0	01	0	1	0	X	0	X
	BEQ	0	00	X	0	01	0	0	0	X	0	1
	BNE	0	00	X	0	01	0	0	0	X	0	1
	FOR	1	10	0	X	10	1	0	0	0	1	0
J	JMP	X	X	X	X	X	0	0	0	X	0	X
	CALL	X	X	X	X	X	0	0	0	X	0	X
	RET	X	X	X	X	X	0	0	0	X	0	X

Main control Boolean Expressions

By tracing the Datapath for each instruction and recording the values of the control signals in a table, we derived Boolean equations for these signals. The control unit implements these equations to generate the appropriate signals for the instruction being executed.

Table3 : Main control unit Boolean expression

Signal	Boolean Expression
call_en	CALL
Src1	AND + ADD + SUB + SLL + SRL
Src2[0]	FOR
Src2[1]	AND + ADD + SUB + SLL + SRL
DesReg	AND + ADD + SUB + SLL + SRL
ExtOP	ANDI
SrcB [0]	ANDI + ADDI + LW + SW + BEQ + BNE
SrcB [1]	For
RegWr	(SW + BEQ + BNE + JMP + CALL + RET) `
MemWr	SW
MemR	LW
WBData	LW

ALU Control Unit Signals

The ALU control unit determines the operation the ALU will perform. It accepts a 4-bit opcode and a 3-bit function as inputs and generates the ALU_Ctrl signal as output, which defines the specific operation to execute within the ALU. Additionally, it produces a zero flag, which is utilized in branch control to decide whether a branch should be executed. The ALU control signals are summarized in the table below:

Table4 :ALU control unit Signals

		OUTPUT	INPUTS	
INST type	INST	ALU ctrl	Opcode	func
R	AND	AND	0000	000
	ADD	ADD	0000	001
	SUB	SUB	0000	010
	SLL	SLL	0000	011
	SRL	SRL	0000	100
I	ANDI	AND	0010	X
	ADDI	ADD	0011	X
	LW	ADD	0100	X
	SW	ADD	0101	X
	BEQ	X	X	X
	BNE	X	X	X
	FOR	SUB	1000	X
J	JMP	X	X	X
	CALL	X	X	X
	RET	X	X	X

ALU Boolean Expressions

Similarly to the main control unit, the Boolean expressions for the ALU control signals were derived and. It is evident that the ALU executes five operations: AND, ADD, SUB, SLL, and SRL**.**

Table5 :ALU control unit expressions

ALU_Operation	Boolean Expression
AND	(Opcode = 0000). (func = 000) + (Opcode = 0010)
ADD	(Opcode = 0000). (func = 001) + (Opcode = 0011) + (Opcode = 0100) + (Opcode = 0101)
SUB	(Opcode = 0000). (func = 010) + (Opcode = 1000)
SLL	(Opcode = 0000). (func = 011)
SRL	(Opcode = 0000). (func = 100)

PC Control Unit Signals

The PC control unit is responsible for managing the address loaded into the PC register during the fetch stage. It takes four inputs: a 4-bit opcode, a 3-bit function, the B_flag, and for_en flag. Its outputs include PCSrc, Call_en, and kill signals. The PCSrc signals (Pcsrc0, Pcsrc1, and Pcsrc2) act as selection lines for a multiplexer that determines the next address to be stored in the PC register. The Call_en signal indicates whether to store the return address to the RR during function calls, while the kill signal serves as a selection line for the multiplexer to decide whether to terminate the current instruction.

Note: The B_flag replaces the traditional ALU Zero flag in the pipeline processor, as branching decisions are now made in the ID stage instead of the ALU.

Table6 :PC control unit signals

			INPUTS			**OUTPUTS**		
INST type	INST	Opcode	func	BFlag	for_en	PCSrc	CALL_en	KILL
R	ALL R-TYPE	0000	X	X	0	000	0	0
I	ANDI	0010	X	X	0	000	0	0
	ADDI	0011	X	X	0	000	0	0
	LW	0100	X	X	0	000	0	0
	SW	0101	X	X	0	000	0	0
	BEQ	0110	X	1	0	001	0	1
	BEQ	0110	X	0	0	000	0	0
	BNE	0111	X	0	0	001	0	1
	BNE	0111	X	1	0	000	0	0
	FOR	1000	X	0	1	100	0	1
J	JMP	0001	000	X	0	010	0	1
	CALL	0001	001	X	0	010	1	1
	RET	0001	010	X	0	011	0	1

PC Boolean Expressions

Boolean equations for these control signals are derived to meet the requirements of various instructions, ensuring accurate branching, function calls, and instruction control.

Table7 : PC control unit Boolean expressions

Signal	**Boolean Expression**
PCSrc [0]	BFlag. (Opcode = 0110. ~for_en) + (~BFlag. (Opcode = 0111). ~for_en) + ((Opcode = 0001). (func = 010). ~for_en)
PCSrc [1]	Opcode = 0001. ~for_en
PCSrc [2]	Opcode = 1000. for_en
CALL_en	(Opcode = 0001). (func = 001). ~for_en
kill	(Opcode = 0110). (BFlag). (~for_en) + (Opcode = 0111). (~BFlag). (~for_en) +(Opcode = 1000). (~BFlag). (for_en) + (Opcode = 0001). (~for_en)

Hazard Unit Boolean Expressions

The Hazard Control Unit addresses data hazards by producing forwarding and stall signals to guarantee accurate instruction execution. The Boolean equations for these forwarding and stall signals are as follows:

Table8 : Hazard unit Boolean expressions

Signal	Boolean Expression
FA = 0	(FA! = 1). (FA! = 2). (FA! = 3)
FA = 1	(Rs1! = 0). (Rs1==Rd1). RegWr1
FA = 2	(Rs1! = 0). (Rs1==Rd2). RegWr2
FA = 3	(Rs1! = 0). (Rs1==Rd3). RegWr3
FB = 0	(FB = 1). (FB! = 2). (FB! = 3)
FB = 1	(Rs2! = 0). (Rs2 == Rd1). RegWr1
FB = 2	(Rs2! = 0). (Rs2 == Rd2). RegWr2
FB = 3	(Rs2! = 0). (Rs2 == Rd3.RegWr3)
STALL	(MemREx = 1). (FA = 1 + FB = 1)

Test Case and Simulation

The following picture shows a sequence of instructions, and the values stored in the data memory.

Figure3 :test case

```
program.dat
4200
4400
0613
0814
```

Figure4 :values in data memory

```
data_mem.dat
1234    // memory[0]
5678    // memory[1]
0000    // memory[2]
0000    // memory[3]
0000    // memory[4]
Ctrl+L to chat, Ctrl+K to generate
```

The following snapshots shows the pipelining mechanism while executing the program above. The first snapshot shows that instruction 1 is being in the fetch stage, whereas in figure2 the first instruction moved to the decode stage and the second instruction is being in the fetch stage. In figure 3, it shows that also worked as pipeline, but it occurs a problem in the execution stage (in the ALU0.

Figure5 simulation snapshots:

```
# KERNEL: Instruction Memory Contents:
# KERNEL: memory[0] = 4200 (LW R1, 0(R0))
# KERNEL: memory[1] = 4400 (LW R2, 0(R0))
# KERNEL: memory[2] = 0613 (SLL R3, R0, R2)
# KERNEL: memory[3] = 0814 (SRL R4, R0, R2)
# KERNEL: Fetching instruction at PC=xxxx: xxxx (Unknown instruction: xxxx)
# KERNEL: Fetching instruction at PC=0000: 4200 (LW R1, 0(R0))
# KERNEL:
# KERNEL: Time=45000 ns: PC=0x0000
# KERNEL:
# KERNEL: Fetch Stage:
# KERNEL: PC = 0000
# KERNEL: Instruction = 4200 (LW R1, 0(R0))
# KERNEL: Next_PC = 0001
# KERNEL:
# KERNEL: Decode Stage:
# KERNEL: Instruction = 0000 (AND R0, R0, R0)
# KERNEL: Rs1 = 0
# KERNEL: Rs2 = 0
# KERNEL: Rdest = 0
# KERNEL: Control Signals:
# KERNEL: RegWr=1, MemR=0, MemW=0
# KERNEL:
# KERNEL: Execute Stage:
# KERNEL: ALU_A = 0000
# KERNEL: ALU_B = 0000
# KERNEL: ALU_Result = 0000
# KERNEL:
# KERNEL: Memory Stage:
# KERNEL: Address = 0000
# KERNEL: MemRead = 0
# KERNEL: MemWrite = 0
# KERNEL: Write_Data = 0000
# KERNEL:
# KERNEL: Writeback Stage:
# KERNEL: WriteBack_Data = 0000
# KERNEL: WriteBack_Reg = 0
# KERNEL: RegWrite = 0
# KERNEL:
# KERNEL: Register File Status:
# KERNEL: R0 = 0x0000
# KERNEL: R1 = 0x0000
# KERNEL: R2 = 0x0000
# KERNEL: R3 = 0x0000
# KERNEL: R4 = 0x0000
# KERNEL: R5 = 0x0000
# KERNEL: R6 = 0x0000
# KERNEL: R7 = 0x0000
# KERNEL:
# KERNEL: Pipeline Registers:
# KERNEL: IF/ID: PC=0000, Inst=0000
# KERNEL: ID/EX: Rs1=0, Rs2=0, Rd=0
# KERNEL: EX/MEM: ALUResult=0000, WriteReg=0
# KERNEL: MEM/WB: WriteData=0000, WriteReg=0
# KERNEL: ----------------------------------------
```

```
# KERNEL:
# KERNEL: Fetching instruction at PC=0001: 4400 (LW R2, 0(R0))
# KERNEL:
# KERNEL: Time=55000 ns: PC=0x0001
# KERNEL:
# KERNEL: Fetch Stage:
# KERNEL: PC = 0001
# KERNEL: Instruction = 4400 (LW R2, 0(R0))
# KERNEL: Next_PC = 0002
# KERNEL:
# KERNEL: Decode Stage:
# KERNEL: Instruction = 4200 (LW R1, 0(R0))
# KERNEL: Rs1 = 0
# KERNEL: Rs2 = 0
# KERNEL: Rdest = 1
# KERNEL: Control Signals:
# KERNEL: RegWr=1, MemR=1, MemW=1
# KERNEL:
# KERNEL: Execute Stage:
# KERNEL: ALU_A = 0000
# KERNEL: ALU_B = 0000
# KERNEL: ALU_Result = 0000
# KERNEL:
# KERNEL: Memory Stage:
# KERNEL: Address = 0000
# KERNEL: MemRead = 0
# KERNEL: MemWrite = 0
# KERNEL: Write_Data = 0000
# KERNEL:
# KERNEL: Writeback Stage:
# KERNEL: WriteBack_Data = 0000
# KERNEL: WriteBack_Reg = 0
# KERNEL: RegWrite = 0
# KERNEL:
# KERNEL: Register File Status:
# KERNEL: R0 = 0x0000
# KERNEL: R1 = 0x0000
# KERNEL: R2 = 0x0000
# KERNEL: R3 = 0x0000
# KERNEL: R4 = 0x0000
# KERNEL: R5 = 0x0000
# KERNEL: R6 = 0x0000
# KERNEL: R7 = 0x0000
# KERNEL:
# KERNEL: Pipeline Registers:
# KERNEL: IF/ID: PC=0001, Inst=4200
# KERNEL: ID/EX: Rs1=0, Rs2=0, Rd=0
# KERNEL: EX/MEM: ALUResult=0000, WriteReg=0
# KERNEL: MEM/WB: WriteData=0000, WriteReg=0
# KERNEL: ----------------------------------------
```

```
# KERNEL:
# KERNEL: Fetching instruction at PC=0002: 0613 (SLL R3, R0, R2)
# KERNEL: Memory Read Operation:
# KERNEL: Address: 0000
# KERNEL: Data: 1234
# KERNEL:
# KERNEL: Time=65000 ns: PC=0x0002
# KERNEL:
# KERNEL: Fetch Stage:
# KERNEL: PC = 0002
# KERNEL: Instruction = 0613 (SLL R3, R0, R2)
# KERNEL: Next_PC = 0003
# KERNEL:
# KERNEL: Decode Stage:
# KERNEL: Instruction = 4400 (LW R2, 0(R0))
# KERNEL: Rs1 = 0
# KERNEL: Rs2 = 0
# KERNEL: Rdest = 2
# KERNEL: Control Signals:
# KERNEL: RegWr=1, MemR=1, MemW=1
# KERNEL:
# KERNEL: Execute Stage:
# KERNEL: ALU_A = 0000
# KERNEL: ALU_B = 0000
# KERNEL: ALU_Result = 0000
# KERNEL:
# KERNEL: Memory Stage:
# KERNEL: Address = 0000
# KERNEL: MemRead = 1
# KERNEL: MemWrite = 1
# KERNEL: Write_Data = 0000
# KERNEL:
# KERNEL: Writeback Stage:
# KERNEL: WriteBack_Data = 0000
# KERNEL: WriteBack_Reg = 0
# KERNEL: RegWrite = 1
# KERNEL:
# KERNEL: Register File Status:
# KERNEL: R0 = 0x0000
# KERNEL: R1 = 0x0000
# KERNEL: R2 = 0x0000
# KERNEL: R3 = 0x0000
# KERNEL: R4 = 0x0000
# KERNEL: R5 = 0x0000
# KERNEL: R6 = 0x0000
# KERNEL: R7 = 0x0000
# KERNEL:
# KERNEL: Pipeline Registers:
# KERNEL: IF/ID: PC=0002, Inst=4400
# KERNEL: ID/EX: Rs1=0, Rs2=0, Rd=1
# KERNEL: EX/MEM: ALUResult=0000, WriteReg=0
# KERNEL: MEM/WB: WriteData=0000, WriteReg=0
# KERNEL: ----------------------------------------


KERNEL: ----------------------------------------
KERNEL:
KERNEL: Memory Write Operation:
KERNEL: Address: 0000
KERNEL: Data: 0000
KERNEL: Memory Read Operation:
KERNEL: Address: 0000
KERNEL: Data: 0000
KERNEL: Fetching instruction at PC=0003: 0814 (SRL R4, R0, R2)
KERNEL:
KERNEL: Time=75000 ns: PC=0x0003
KERNEL:
KERNEL: Fetch Stage:
KERNEL: PC = 0003
KERNEL: Instruction = 0814 (SRL R4, R0, R2)
KERNEL: Next_PC = 0004
KERNEL:
KERNEL: Decode Stage:
KERNEL: Instruction = 0613 (SLL R3, R0, R2)
KERNEL: Rs1 = 0
KERNEL: Rs2 = X
KERNEL: Rdest = 3
KERNEL: Control Signals:
KERNEL: RegWr=1, MemR=0, MemW=0
KERNEL:
KERNEL: Execute Stage:
KERNEL: ALU_A = 0000
KERNEL: ALU_B = xxxx
KERNEL: ALU_Result = 0000
KERNEL:
KERNEL: Memory Stage:
KERNEL: Address = 0000
KERNEL: MemRead = 1
KERNEL: MemWrite = 1
KERNEL: Write_Data = 0000
KERNEL:
KERNEL: Writeback Stage:
KERNEL: WriteBack_Data = 0000
KERNEL: WriteBack_Reg = 0
KERNEL: RegWrite = 1
KERNEL:
KERNEL: Register File Status:
KERNEL: R0 = 0x0000
KERNEL: R1 = 0x0000
KERNEL: R2 = 0x0000
KERNEL: R3 = 0x0000
KERNEL: R4 = 0x0000
KERNEL: R5 = 0x0000
KERNEL: R6 = 0x0000
KERNEL: R7 = 0x0000
KERNEL:
KERNEL: Pipeline Registers:
KERNEL: IF/ID: PC=0003, Inst=0613
KERNEL: ID/EX: Rs1=0, Rs2=X, Rd=2
KERNEL: EX/MEM: ALUResult=0000, WriteReg=1
KERNEL: MEM/WB: WriteData=0000, WriteReg=0
KERNEL: ----------------------------------------
```

We constructed testbenches to all modules to figure out the problem. As shown in the following figure there exist a problem in the ALU, it does not differentiate between the AND & ADD operations.

Figure6 :

```
# KERNEL: 20000 AND 000 Src1=1 Src2=01 DestReg=1 RegWr=1 ExOp=x ALU=000
# KERNEL: 30000 ADD 001 Src1=1 Src2=01 DestReg=1 RegWr=1 ExOp=x ALU=000
```

Also, we verified that control signals were propagated properly through all the stages as shown below.

Figure7 :

```
# KERNEL: Pipeline Register Values:
# KERNEL: Time DE Stage   EMem Stage  MemWb Stage
# KERNEL: ------------------------------------------------------------
# KERNEL: 140000 RegWr=0 MemR=0 MemW=0 RegWr=0 MemR=0 MemW=0 RegWr=1
```

Teamwork

1. RTL Design

Collaborative Effort: All members contributed to brainstorming and Datapath design.

RTL Division:

- Amir: R-type instructions.
- Joud: I-type instructions.
- Labiba: J-type and branch instructions.

2. Datapath design, Verilog Implementation and Testing

- Joud and Labiba: Instruction Fetch and Decode.
- Amir: Execution, Memory Access, and Write Back.

The team agreed on implementing single cycle code gradually by adding each stage with and then implementing same approach for pipelined code.

- Joud and Labiba: Instruction Fetch and Decode, control units' codes and testbenches.
- Amir: Execution, Memory Access, and Write Back, hazard unit codes and testbenches.

3. Simulation

We were all working together in this part.

4. Report Writing

- Labiba: Design and Implementation and conclusion.
- Joud: Abstract, Introduction, teamwork, and Control Signals.
- Amir: Control signals and reviewed the report for accuracy and style.

Conclusion

In this project, we aimed to design and verify a simple pipelined RISC processor in Verilog. The initial design phase, including the Datapath and control path development, was successfully completed. We also implemented all required components to support the given ISA, examining to the project specifications.

However, the testing and simulation phase did not yield successful results. Despite multiple iterations of debugging and verification efforts, the processor did not function as intended in the testbench environment. Key issues likely stem from errors in the input and clock delays in the testing programs as well as the errors in the execution stage. These shortcomings prevented us from validating the processor's execution of the instruction set and achieving the desired outcomes.